ARCHITECTURE & ENGINEERING GUIDE

The Complete Guide to gRPC

A comprehensive handbook covering Remote Procedure Calls, Protocol Buffers binary serialization, HTTP/2 multiplexing, real-world C# .NET implementation, and production architecture patterns.

Protocol: gRPC v1.x (Google / CNCF) **Format:** Protocol Buffers v3 **Transport:** HTTP/2 Full-Duplex **Framework:** .NET 8 / 9 / 10 & Polyglot

1 What is gRPC?

gRPC (which stands for *gRPC Remote Procedure Calls*) is a high-performance, open-source universal RPC framework created by **Google** in 2015 and now governed by the **Cloud Native Computing Foundation (CNCF)**.

In traditional REST architectures, services exchange text-based JSON or XML documents over HTTP/1.1 endpoints. With gRPC, a client application can **directly invoke a method on a remote server application as if it were a local in-memory method call**, regardless of whether the server is located on the same host or across the globe, and regardless of what language the server is written in.

High Performance & Low Latency

By using compact binary serialization (Protocol Buffers) instead of verbose JSON strings, payloads are up to 70% smaller, and serialization is 5x to 8x faster with drastically lower CPU overhead.

Strict Contract-First Safety

Services and schemas are defined in a language-agnostic `.proto` contract. Client and server stubs are automatically generated at compile time, eliminating typos and runtime serialization bugs.

Polyglot Architecture

Write your backend in C#, your AI model service in Python, and your high-throughput gateway in Go or Rust. All services share the exact same `.proto` contracts effortlessly.

True Bidirectional Streaming

Built directly on HTTP/2, gRPC provides native support for request streaming, response streaming, and simultaneous full-duplex bidirectional streaming over a single TCP connection.

2 Why Was gRPC Invented?

Inside Google, massive distributed clusters communicate billions of times per second. Traditional REST/JSON caused three major bottlenecks:

1. **JSON Parsing Overhead:** Converting ASCII text strings into memory objects consumes significant CPU and memory allocation cycles.
2. **HTTP/1.1 Connection Head-of-Line Blocking:** Browsers and servers had to open 6+ separate TCP connections to send concurrent requests, consuming socket descriptors and handshakes.
3. **Contract Drift:** Without strict schemas, REST APIs frequently drifted from client expectations, leading to runtime null-reference and schema mismatches.

gRPC solved all three problems by combining **Protocol Buffers** (binary serialization) with **HTTP/2** (multiplexed streams).

3 The 4 Architectural Pillars of gRPC

1. Protocol Buffers (Protobuf) – The Binary Contract

Protobuf is Google's language-neutral mechanism for serializing structured data. Instead of serializing human-readable field names as strings (e.g. `{"name": "Jaydeep"}`), Protobuf encodes fields into tiny binary key-value tags where numbers represent fields:

```
// greet.proto
syntax = "proto3";
option csharp_namespace = "GrpcService";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string name = 1;  // '1' is the unique binary field tag, NOT a value!
}

message HelloReply {
  string message = 1;
}
```

2. HTTP/2 Transport Layer

Unlike HTTP/1.1, gRPC runs exclusively over **HTTP/2** as its transport protocol:

- **Multiplexing:** Hundreds of independent bidirectional calls stream over a *single persistent TCP connection* simultaneously without blocking.
- **HPACK Header Compression:** Compresses repetitive metadata headers, dramatically cutting network bandwidth.
- **Flow Control & Reset:** Fine-grained control over byte streaming without tearing down the entire connection.

3. Automatic Code Generation (Stubs)

The compiler (`protoc` or .NET's `Grpc.Tools`) compiles `greet.proto` into strongly-typed C# classes:

- **Server Base Class:** Provides an abstract class (`Greeter.GreeterBase`) that you inherit and override with your business logic.
- **Client Stub:** Provides a ready-to-use client class (`Greeter.GreeterClient`) with async methods matching every RPC endpoint.

4. Streaming Capabilities (4 Communication Models)

RPC Type	Description	Typical Real-World Use Case
Unary RPC	Single request from client → Single response from server (Standard Request/Response).	Standard CRUD, Authentication, User profile lookup.
Server Streaming	Single request → Stream of multiple response messages sent by the server over time.	Live market data tickers, downloading large files in chunks, real-time log viewers.
Client Streaming	Stream of messages from client → Single summary response from server.	IoT telemetry upload, uploading large video files, batch sensor logging.
Bidirectional Streaming	Both client and server stream read/write messages independently and concurrently.	Real-time chat applications, multiplayer game synchronization, live video/audio conferencing.

4 How gRPC Works Under the Hood (Execution Flow)



Here is what happens during a single gRPC call in detail:

1. **Client Invocation:** The client application calls `await client.SayHelloAsync(request)` as if it were a standard local C# method.
2. **Client Stub Serialization:** The generated client stub intercepts the call. It feeds the C# object into the Protobuf serializer, converting properties into compact binary format.
3. **HTTP/2 Frame Dispatch:** The stub wraps the payload in an HTTP/2 `DATA` frame with metadata headers (including `content-type: application/grpc` and target path `/package.Service/Method`). It dispatches this frame across the established TCP multiplexed connection.
4. **Server Deserialization:** The ASP.NET Core Kestrel HTTP/2 server receives the frame, identifies the route, and passes the binary payload to the Protobuf deserializer, which reconstructs the strongly typed C# parameter.
5. **Business Logic Execution:** The server executes the overridden `SayHello` method in `GreeterService`.
6. **Return Trip:** The return object is serialized into binary, sent back as an HTTP/2 response frame along with gRPC status trailers (e.g. `grpc-status: 0 (OK)`), and unpacked into the client's return object.

5 Practical C# .NET Implementation Walkthrough

Below is how the server and client interact in our solution:

1. Server Service (GreeterService.cs)

```
public class GreeterService : Greeter.GreeterBase
{
    public override Task<HelloReply> SayHello(
        HelloRequest request, ServerCallContext context)
    {
        // Read parameter & return reply object
        return Task.FromResult(new HelloReply
        {
            Message = "Hello " + request.Name
        });
    }
}
```

2. Client Consumer (Program.cs)

```
// Reuse one channel across application lifetime!
using var channel = GrpcChannel.ForAddress(
    "http://localhost:5273");

var client = new Greeter.GreeterClient(channel);

// Strongly-typed remote procedure call:
var reply = await client.SayHelloAsync(
    new HelloRequest { Name = "Robin" });

Console.WriteLine(reply.Message);
```

6 When to Use gRPC (and When NOT to Use It)

✅ Best Used For (Ideal Scenarios)

- **Internal Microservices (East-West Traffic):** Service-to-service communication within Kubernetes or private clouds where latency and throughput are critical.
- **Polyglot Systems:** Environments where microservices are written in diverse languages (C#, Go, Python, Java) needing a single contract source of truth.
- **Real-Time Streaming Systems:** Applications needing continuous data feeds, live market rates, GPS location tracking, or telemetry.
- **Low-Power & Mobile Devices (IoT):** Compact binary payloads consume significantly less bandwidth and battery life.

❌ NOT Recommended For

- **Direct Browser Front-ends (Public Web APIs):** Browsers do not expose low-level control of HTTP/2 framing. Connecting browsers to gRPC requires a proxy (such as Envoy and gRPC-Web).
- **Public Third-Party APIs:** External developers expect standard REST/JSON endpoints easily consumable via curl, Postman, or Swagger without needing Protobuf compilers.
- **Edge Caching:** HTTP/1.1 REST responses leverage standard reverse-proxy caching (Varnish, Cloudflare, CDN). gRPC uses HTTP POST for all calls, making edge caching complex.

7 gRPC vs REST vs GraphQL vs WebSockets

Feature	gRPC	REST (JSON)	GraphQL	WebSockets
Protocol	HTTP/2	HTTP/1.1 or HTTP/2	HTTP/1.1 or HTTP/2	TCP (WS Handshake)
Data Format	Binary (Protocol Buffers)	Text (JSON, XML)	Text (JSON)	Text / Binary Frames
Contract Style	Strict (`.proto` file)	Optional (OpenAPI/Swagger)	Strict (GraphQL Schema)	None (Custom protocol)
Performance	Ultra Fast (Low CPU)	Moderate	Moderate	Fast
Streaming	Full Bidirectional	No (SSE one-way only)	Subscriptions (via WS)	Full Bidirectional
Browser Support	Requires gRPC-Web proxy	Native & Universal	Native & Universal	Native & Universal
Primary Use Case	Backend Microservices, IoT	Public APIs, Web CRUD	Mobile & Web frontends	Real-time chat, gaming

8 Production Best Practices for gRPC

1. Reuse GrpcChannel Instances

Creating a `GrpcChannel` is expensive because it establishes TLS and HTTP/2 handshakes. **Never create a channel per call.** Keep it as a singleton or inject it via `services.AddGrpcClient<T>()` .

2. Always Set Deadlines (Timeouts)

Network calls can hang indefinitely without timeouts. Always pass a deadline:

```
await client.SayHelloAsync(request, deadline: DateTime.UtcNow.AddSeconds(5));
```

3. Schema Evolution Guidelines

Never change field tag numbers (e.g., `string name = 1;`). If you deprecate a field, mark it as `reserved 1;` so future developers do not reuse the slot and break backwards compatibility.

4. Use Layer 7 Load Balancing

Because gRPC runs over long-lived multiplexed HTTP/2 TCP streams, traditional L4 load balancers send all traffic to just one pod. Use an L7 proxy (e.g. Envoy, Traefik, or Nginx) to balance individual RPC calls.

Executive Rule of Thumb: Use **gRPC** when you control both the client and server (backend microservices, real-time pipelines, or mobile apps) for maximum throughput, compile-time safety, and minimal latency. Use **REST** when building public APIs meant for external consumption across the web.